

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – MINI APPLICATION DEVELOPMENT (CHATBOT /
SUMMARIZER / TRANSLATOR)

Course Code:	CSA65	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	Assessment:	Assessment Tool 1-Mini Application Development (Chatbot / Summarizer / Translator)
Weightage:	20% of CIA	Total Marks:	2 * 50= 100
CO4 Statement:	<i>Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.(BL3, BL4,BL5)</i>		
Student Name:	J KARTHIK	Reg. No.:	192472136
Section / Batch:	CSE – AI / 2024	Date:	

Instructions to Students

Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.

Implement the solution using **Python**.

Select and justify an appropriate AI/LLM model or technique.

Develop a **working application with a user interface**.

Test the application using multiple input cases.

Demonstrate handling of invalid, empty, or unexpected inputs.

Clearly explain the application architecture and workflow.

Submit the source code, test results, screenshots/demo, and documentation.

Explain the limitations of the selected model/technique and suggest possible improvements.

Code of Conduct

I J KARTHIK / 192472136(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: J KARTHIK

QUESTION 1 – TEXT SUMMARIZATION APPLICATION

Question No. 15 – News Article Summarizer (50 Marks)

Source question: Develop a text summarization application that accepts a long news article and generates a concise summary. Provide a simple interface and allow the user to control the approximate summary length.

1. Aim

To develop a Python-based AI news article summarizer that accepts a long article, generates a concise abstractive summary, and allows the user to select an approximate summary length through a simple graphical web interface.

2. Objectives

- Accept a long news article as text input.
- Generate a meaningful concise summary using a pretrained language model.
- Provide short, medium, and detailed summary-length options.
- Validate empty, very short, and unexpected input.
- Display the generated summary in a user-friendly interface.
- Test the application with multiple input cases.

3. Selected AI/LLM Model and Justification

Selected model: facebook/bart-large-cnn (BART abstractive summarization model).

- It is designed for sequence-to-sequence text generation and summarization.
- It can produce abstractive summaries rather than only copying sentences.
- It is suitable for news-style English articles.
- A pretrained model avoids training a summarizer from scratch.
- The model can be integrated directly with the Hugging Face Transformers pipeline.

Technique: Abstractive summarization. The model understands the input sequence and generates a shorter version containing the major information.

4. System Architecture

User → Streamlit UI → Input Validation → BART Summarization Model → Length Control → Summary Display

- Presentation layer: Streamlit text area, length selector, and buttons.
- Validation layer: checks empty input and minimum useful content.
- AI layer: BART tokenizer/model processes the article and generates the summary.
- Output layer: displays the generated summary and basic statistics.
- Error layer: catches unexpected processing/model errors and shows a readable message.

5. Workflow

- Start the Streamlit application.
- Enter or paste a news article.
- Select Short, Medium, or Detailed summary.
- Click Generate Summary.
- Validate the input.
- Tokenize the article and pass it to the BART model.
- Generate the summary using selected length parameters.
- Decode the generated tokens into readable text.
- Display the summary and summary length.

6. Required Installation

pip install streamlit transformers torch sentencepiece

7. Source Code – app_summarizer.py

```
import streamlit as st
from transformers import pipeline

st.set_page_config(page_title="News Article Summarizer", page_icon="")
st.title(" AI News Article Summarizer")
st.write("Paste a long news article and generate a concise summary.")

@st.cache_resource
def load_model():
    return pipeline("summarization", model="facebook/bart-large-cnn")

summarizer = load_model()

length = st.selectbox("Summary Length", ["Short", "Medium", "Detailed"])
article = st.text_area("Enter News Article", height=280)

limits = {
    "Short": (35, 60),
    "Medium": (60, 100),
    "Detailed": (100, 150)
}

if st.button("Generate Summary"):
    text = article.strip()

    if not text:
        st.error("Invalid input: Please enter a news article.")
    elif len(text.split()) < 30:
        st.warning("Please provide a longer article (at least 30 words).")
```

```

else:
    min_len, max_len = limits[length]

    try:
        # BART accepts limited input length, so keep the demo stable.
        words = text.split()
        text = " ".join(words[:900])

        result = summarizer(
            text,
            min_length=min_len,
            max_length=max_len,
            do_sample=False
        )

        st.subheader("Generated Summary")
        st.write(result[0]["summary_text"])

    except Exception as e:
        st.error("Unexpected processing error. Please try again.")
        st.caption(str(e))

```

8. Sample Input and Output

Sample Input:

The city administration announced a new public transport improvement programme on Monday. The programme will introduce additional electric buses, upgrade bus stops, improve digital ticketing, and provide better route information for passengers. Officials said the project is intended to reduce traffic congestion, lower emissions, and make public transport more convenient. The first phase will begin next month in major residential and commercial areas. The administration also stated that passenger feedback will be collected during the initial phase and used to improve routes and schedules.

Selected length: Short

Sample Output:

The city administration announced a public transport improvement programme with more electric buses, upgraded bus stops, digital ticketing and better route information. The project aims to reduce congestion and emissions, with the first phase beginning next month.

9. Test Cases and Results

Test Case	Input	Expected / Observed Result
TC1	Valid long news article	Concise summary generated successfully.
TC2	Same article + Medium	Longer summary generated.
TC3	Same article + Detailed	More detailed summary generated.
TC4	Empty input	Error: Please enter a news article.
TC5	Very short input	Warning requesting at least 30 words.
TC6	Unexpected processing/model error	Exception handled and readable error displayed.

10. Invalid and Unexpected Input Handling

- Empty input: rejected before model execution.
- Very short input: warning is displayed because meaningful summarization is unlikely.
- Excessively long input: the demo limits the text passed to the model to maintain stable execution.
- Model/runtime failure: handled using try-except and an error message.
- Whitespace-only input: strip() converts it to an empty input and rejects it.

11. Advantages

- Simple user interface.
- Uses a pretrained AI model.
- Supports different summary lengths.
- Requires no model training.
- Provides input validation and error handling.

12. Limitations

- BART-large-CNN is mainly intended for English news-style text.
- Long documents may exceed the model input limit.
- Generated summaries may occasionally omit details or introduce inaccurate wording.
- Inference can require significant RAM/CPU/GPU resources.
- The application depends on the availability of the pretrained model.

13. Possible Improvements

- Use a long-document summarization model for very large articles.
- Add PDF and DOCX upload support.
- Add multilingual summarization.
- Add summary quality checks and factual consistency evaluation.
- Allow the user to download the summary as a text/PDF file.

14. Conclusion

The News Article Summarizer satisfies the selected assessment requirement by accepting a long article, generating a concise AI-based summary, providing summary-length control, and handling invalid inputs through a Python Streamlit interface.

QUESTION 2 – MACHINE TRANSLATION APPLICATION

Question No. 28 – English-to-Tamil Translation Application (50 Marks)

Source question: Develop a machine translation application that translates English text into Tamil. Provide a user-friendly interface and demonstrate the system using different types of sentences.

1. Aim

To develop a Python-based English-to-Tamil machine translation application that accepts English text and produces Tamil translation through a simple interactive web interface.

2. Objectives

- Accept English sentences as input.
- Translate the input into Tamil using a pretrained neural machine translation model.
- Provide a simple and user-friendly interface.
- Test simple, complex, and conversational sentences.
- Handle empty, unsupported, and unexpected inputs.
- Display the translated result clearly.

3. Selected AI/LLM Model and Justification

Selected model: Helsinki-NLP/opus-mt-en-ta.

- It is a pretrained neural machine translation model for English-to-Tamil translation.
- It avoids the need to train a translation model from scratch.
- It is appropriate for the selected language direction.
- It can be integrated using the Hugging Face Transformers pipeline.
- The model is suitable for a compact educational mini-application.

Technique: Neural Machine Translation (NMT) using a pretrained sequence-to-sequence Transformer model.

4. System Architecture

User → Streamlit UI → Input Validation → English-to-Tamil NMT Model → Tamil Output

- UI layer: text area and Translate button.
- Validation layer: checks for empty input.
- Translation layer: tokenizes English text and generates Tamil output.
- Output layer: displays the translated sentence.
- Error layer: handles unexpected model/runtime failures.

5. Workflow

- Start the Streamlit application.
- Enter an English sentence.

- Click Translate to Tamil.
- Validate the input.
- Pass valid text to the pretrained NMT model.
- Generate Tamil tokens.
- Decode tokens into Tamil text.
- Display the translated result.

6. Required Installation

`pip install streamlit transformers torch sentencepiece`

7. Source Code – `app_translator.py`

```
import streamlit as st
from transformers import pipeline

st.set_page_config(page_title="English to Tamil Translator", page_icon=" ")
st.title(" English → Tamil Translator")
st.write("Enter an English sentence and translate it into Tamil.")

@st.cache_resource
def load_model():
    return pipeline(
        "translation",
        model="Helsinki-NLP/opus-mt-en-ta"
    )

translator = load_model()
text = st.text_area("Enter English Text", height=180)

if st.button("Translate to Tamil"):
    text = text.strip()

    if not text:
        st.error("Invalid input: Please enter English text.")
    else:
        try:
            words = text.split()
            if len(words) > 100:
                st.warning("Input is long; only the first 100 words will be translated.")
                text = " ".join(words[:100])

            result = translator(text, max_length=256)

            st.subheader("Tamil Translation")
            st.write(result[0]["translation_text"])
```

```
except Exception as e:  
    st.error("Unexpected translation error. Please try again.")  
    st.caption(str(e))
```

8. Sample Inputs and Outputs

Test Case 1 – Simple sentence

Input:

I am going to college today.

Sample Output:

நான் இன்று கல்லூரிக்குச் செல்கிறேன்.

Test Case 2 – Complex sentence

Input:

Students should complete their assignments before the examination.

Sample Output:

மாணவர்கள் தேர்வுக்கு முன் தங்கள் பணிகளை முடிக்க வேண்டும்.

Test Case 3 – Conversational sentence

Input:

Can you help me with this problem?

Sample Output:

இந்தப் பிரச்சினையில் எனக்கு உதவ முடியுமா?

Note: Exact wording can vary slightly depending on the pretrained model version and runtime.

9. Test Cases and Results

Test Case	Input Type	Expected / Observed Result
TC1	Simple English sentence	Tamil translation generated.
TC2	Complex English sentence	Tamil translation generated while preserving main meaning.
TC3	Conversational sentence	Tamil conversational translation generated.
TC4	Empty input	Error message displayed.
TC5	Whitespace-only input	Rejected as empty input.
TC6	Very long input	Warning shown and input limited for stable execution.
TC7	Unexpected runtime/model error	Exception handled and readable error displayed.

10. Invalid and Unexpected Input Handling

- Empty input is rejected.
- Whitespace-only input is treated as empty input.
- Very long input is limited to reduce runtime and memory problems.
- Unexpected model/runtime failures are caught using try-except.
- The interface provides a clear error or warning instead of terminating the application.

11. Advantages

- Easy-to-use interface.
- Direct English-to-Tamil translation.
- Uses a pretrained neural translation model.
- No manual dictionary or rule-based translation is required.
- Suitable for an educational demonstration.

12. Limitations

- Translation quality depends on the pretrained model.
- Idioms, slang, ambiguity, and domain-specific terminology may not translate perfectly.
- Very long inputs can increase latency and memory usage.
- The model may produce grammatically acceptable but contextually imperfect translations.
- The model is specific to the English-to-Tamil direction.

13. Possible Improvements

- Add automatic language detection.
- Support Tamil-to-English and other Indian languages.
- Add translation history and downloadable output.
- Add domain-specific terminology support.
- Use a stronger multilingual translation model where appropriate.
- Add human evaluation or automatic translation-quality metrics.

14. Conclusion

The English-to-Tamil Translation Application satisfies the selected assessment requirement by providing an interactive Python interface, using a pretrained neural machine translation model, demonstrating multiple sentence types, and handling invalid or unexpected inputs.

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Model/Technique Fit	30%	Correct, appropriate model/technique choice for the chosen application	Mostly appropriate choices made	Some inappropriate technique choices	Technique choice fundamentally flawed
Functional Completeness	25%	Application fully functional; solves a real use-case effectively	Mostly functional application	Partially functional application	Application is non-functional
Robustness & Reliability	20%	Handles edge cases; stable, reliable performance	Handles common cases well	Limited robustness to edge cases	Frequent failures during use
UI / Demo Polish	15%	Polished UI/demo with an engaging walkthrough	Clear demo, minor polish issues	Basic or rough demo	No usable demo presented
Design & Usage Documentation	10%	Comprehensive documentation of design & usage	Adequate documentation	Minimal documentation	No documentation provided

Marks Evaluation:

Question No.	Model/Technique Fit (30%)	Functional Completeness (25%)	Robustness & Reliability (20%)	UI / Demo Polish (15%)	Design & Usage Documentation (10%)	Total Marks (100)
--------------	---------------------------	-------------------------------	--------------------------------	------------------------	------------------------------------	-------------------

Q1						
Q2						
Overall Total (100)						